Тестовое задание: разработка RAG-системы по руководству насосов WARMAN 4 DY-AHF

1. Контекст

Мы ищем стажёра, который умеет превращать неструктурированные технические документы в удобный интерфейс вопросов-ответов на базе Retrieval-Augmented Generation (RAG). В реальных проектах вам придётся быстро строить прототипы, обеспечивать воспроизводимость экспериментов и доводить их до продакшн-качества.

2. Задача

Создайте Jupyter-ноутбук, который:

1. **Принимает** PDF-документ «Руководство по насосам шламовым 4 DY-AHF».
2. **Строит** индекс (векторный, структурированный или гибридный) с помощью `llama-index` **или** `langchain` (можно комбинировать).
3. **Отвечает** на произвольные вопросы пользователя на естественном языке, опираясь исключительно на содержимое руководства.
4. **Возвращает**:
   - текстовый ответ;
   - ссылки на использованные фрагменты (цитаты, номера страниц) и/или изображение, если оно повышает понятность;
   - сообщение «Не знаю», если информация отсутствует в документе;
   - сообщение о не-релевантности, если вопрос не относится к теме руководства о насосе.

3. Минимальные (обязательные) требования

1. **Точность** и **полнота** ответов на контрольный набор вопросов (см. § 8).
2. **Воспроизводимость**: ноутбук должен выполняться «в один клик» после `git clone` без ручного ввода, кроме API-ключей.
3. **Чёткая структура**: обработка → индексирование → генерация ответа → демонстрация примеров.
4. **Цитирование**: укажите источник (страница/координаты) для каждого фрагмента; если добавляете изображение, приложите его превью или ссылку.
5. **Честность**: при отсутствии данных ответ должен содержать явное признание «Информация не найдена».
6. **Код-стиль**: PEP 8, аннотации типов, docstrings.
7. **README** с инструкцией запуска, переменными окружения и примером запроса.
8. **requirements.txt** с минимальным набором зависимостей, зафиксированных версиями.

4. Опциональные (бонус) задачи

- REST API (FastAPI или Flask) поверх пайплайна.
- Минималистичный веб-интерфейс (Streamlit, Gradio, Panel, etc.).
- Dockerfile + docker-compose для локального развертывания.
- Поддержка изображений (OCR, извлечение фигур) и таблиц.
- Скрипт для расчёта метрик ответов (Exact Match, F1, Rouge-L).
- Логирование и конфигурация через `.env` или `pydantic Settings`.

5. Исходные данные

Файл	Размер	Содержание
`Руководство по насосам шламовым 4 DY-AHF.pdf`	~2 MB	Текст, таблицы, схемы, изображения

6. Рекомендуемый стек

- Python, JupyterLab/Colab/Kaggle
- `llama-index` или `langchain`
- Векторное хранилище: Chroma, Qdrant или аналогичное
- LLM: GPT-4o, Llama-3 8B-Instruct или другая совместимая модель
- PDF-парсинг: pdfplumber, PyPDF2, unstructured
- OCR: pytesseract (по необходимости)

Использование другой библиотеки допустимо при наличии аргументации в README.

7. Структура репозитория

```
├── notebook.ipynb        # основной ноутбук (обязателен)
├── requirements.txt      # pinned dependencies
├── README.md             # инструкция запуска
├── Dockerfile            # опционально
├── docker-compose.yml    # опционально
├── app/                  # API / UI (опционально)
└── tests/                # опционально
```

8. Контрольные вопросы (для самопроверки)

1. Какой объём смазки (л) требуется для подшипниковой рамы **DY**?
2. Укажите кинематическую вязкость рекомендованного масла при **40 °C** для роликовых подшипников.
3. Назовите заводскую смазку, рекомендованную для камеры центробежного уплотнения.
4. Приведите номер телефона сервисной линии для клиентов в **Финляндии**.
5. Какова ширина ремня (мм), указанная для диапазона шкивов **170–224 мм (профиль SPB)**?

6. Перечислите три основных требования техники безопасности при подъёме насоса с вертикальным приводом (CV).
7. Опишите процедуру проверки уровня масла в подшипниковом узле.
8. Какой класс **NLGI** указан для пластичной смазки камеры центробежного уплотнения?
9. Какой индекс вязкости **DIN-ISO 2909** указан для рекомендуемого масла подшипников?
10. На сколько литров объём смазки рамы **FFY** превышает объём рамы **BY**?
11. Согласно схеме **LD002-RUS WP8**, какой тип стропа рекомендован, чтобы предотвратить проскальзывание насоса при подъёме?
12. Сформируйте краткую инструкцию «Ежедневное ТО + подъём» (≤ 100 слов) и приложите соответствующую иллюстрацию.

9. Порядок сдачи

1. Загрузите код на **GitHub / GitLab** в публичный или приватный (с доступом reviewer@company.com) репозиторий.
2. Укажите в README минимальные требования к окружению и команду(ы) для запуска.
3. Отправьте ссылку на репозиторий и PDF-отчёт, сгенерированный ноутбуком (File → Export → PDF) до 18.*03.2026 23:59 UTC+3*.
4. Вы можете использовать как проприетарные, так и open-source модели; при применении платных API обратите внимание на стоимость

Удачи!